Attack Detection on the Software Defined Networking Switches

Uday Tupakula, Vijay Varadharajan, Kallol Krishna Karmakar
Advanced Cyber Security Engineering Research Centre
The University of Newcastle, Newcastle, Australia
[uday.tupakula, vijay.varadharajan, kallolkrishna.karmakar]@newcastle.edu.au

***Abstract*—Software Defined Networking (SDN) is disruptive networking technology which adopts a centralised framework to facilitate fine-grained network management. However security in SDN is still in its infancy and there is need for significant work to deal with different attacks in SDN. In this paper we discuss some of the possible attacks on SDN switches and propose techniques for detecting the attacks on switches. We have developed a Switch Security Application (SSA) for SDN Controller which makes use of trusted computing technology and some additional components for detecting attacks on the switches. In particular TPM attestation is used to ensure that switches are in trusted state during boot time before configuring the flow rules on the switches. The additional components are used for storing and validating messages related to the flow rule configuration of the switches. The stored information is used for generating a trusted report on the expected flow rules in the switches and using this information for validating the flow rules that are actually enforced in the switches. If there is any variation to flow rules that are enforced in the switches compared to the expected flow rules by the SSA, then, the switch is considered to be under attack and an alert is raised to the SDN Administrator. The administrator can isolate the switch from network or make use of trusted report for restoring the flow rules in the switches. We will also present a prototype implementation of our technique.**

***Index Terms*—Software Defined Networking (SDN) Security, Security Attacks, Switch Security, Security Application.**

I. Introduction

Network management is a critical process which involves configuring and monitoring the network devices using cost effective methods. Traditionally, this has been a painstaking process involving manual configuration of individual devices across the network topology. Furthermore, enterprise network management is becoming an increasingly complicated process with rapid migration of services to hybrid cloud environments and users accessing these services with mobile devices.

Software Defined Networking (SDN) is an innovative architectural approach to modern computer networks where the control features of the infrastructure are abstracted from the network devices themselves and placed into a centralized location. This abstraction of the network allows for novel approaches to network management, including third-party applications, dynamic and adaptive configuration, and cloud-hosting. In particular, the applications developed for SDN Controller add value in ways that would have been difficult or impractical before. For instance, SDN facilitates rapid development of software solutions for a wide range of network issues in traditional network architectures such as static configuration, non-scalability and low efficiency. Hence, SDN is becoming immensely popular for network management among modern enterprise networks, cloud and network service providers.

978-1-7281-5684-2/20/$31.00

Although SDN has several benefits, security in SDN is still in its infancy and different attacks are possible in such networks [1], [2], [3]. In particular, the SDN architecture has a larger attack surface than traditional networks due to reasons such as single point failure of the controller, exposed network-wide resources, and the possibility of malicious applications severely disrupting network operations. The devices in the data plane are also vulnerable to different attacks since they are connected to control plane and the data plane. Hence, attacks are possible from malicious applications in the control plane, exploiting the weak or lack of secure communication between the devices, and from end hosts that are connected to the switches. There is ongoing work [4], [5] on enhancing security in SDN but still there is need for significant research for addressing several security issues and developing robust solutions to deal with the attacks. Hence, the aim of this work is to develop techniques for detecting attacks on the switches for enhancing security in SDN. In this paper we first present an attacker model for the switches. Then we propose and develop a Switch Security Application (SSA) for SDN Controllers to detect attacks on switches.

The paper is organised as follows. Section II presents the attacker model for our work. In Section III we propose the Switch Security Application to deal with the attacks on SDN Switches. We first present a high level overview of the SSA and then describe the important components of SSA in detail. In section IV we present the implementation of SSA and demonstrate its usage by considering an example attack case scenario and how SSA can help to detect such attacks. Then we present some performance results. Section V presents general discussion related to our approach. Section VI concludes the paper.

II. Attacker Model

Let us discuss the attacker model for our work. In this paper we consider attacks on the switches. Switches are the critical components that are connected to the Controller in the control plane and end hosts in the data plane. Hence, they are vulnerable to attacks from devices in both planes. Let

us discuss some of the entry points for attackers to generate attacks on the switches.

- The controller often runs a number of applications, each of which includes specific policies and enforces the corresponding rules to be deployed. Often multiple applications can initiate commands to simultaneously manage the same physical network. In many cases, these applications can be designed by third parties that can be buggy or malicious. The current Controllers do not validate the commands initiated by the applications [2]. Hence it is easy for an attacker to make use of malicious applications to generate attacks on the switches.
- Although SSL/TLS based secure communication is supported in the switches, it is not enabled by default. Hence it is a trivial task for attackers to send malicious flow˙mod messages with a spoofed identity of the Controller to alter the flow rules in the switches. Even if there is SSL/TLS based secure communication between the switches, vulnerabilities in SSL/TLS lead to attacks such as Heartbleed which can expose the keys used for securing the communication between the devices. The administrators can update the devices when the patches become available to the vulnerabilities. However, still there is need for the administrators to determine if there are any successful attacks when the devices were vulnerable and before the patches are applied to the devices.
- Since the switches are directly connected to end hosts, the attackers can make use of end hosts to inject malicious flow rules or attackers can exploit a vulnerability in the switch to obtain unauthorised access and alter the existing rules by perhaps changing the deny actions to permit, changing permit actions to deny or providing priority to the flows generated by attacker. Such attacks are easily possible in cloud environments [6]. A more detailed discussion on the attacks on virtual switches can be found in [6].

From the above discussion it is clear that there are different entry points for the attackers to generate these attacks and it is not an easy task for the SDN administrators to determine the entry point or source of the attacks. However, it is still very useful to detect such attacks on the switches. Hence, the main focus in this paper is to detect attacks on the switches.

III. Switch Security Application

In this section, we will first discuss some of the assumptions in our model. Then we present the operation of the SSA logical architecture and describe each component in detail.

- We assume that the Controller in the SDN domain is not compromised. This is a common assumption for security models in SDN.
- We assume that all the switches (physical and virtual) in the SDN domain are equipped with a TPM chip [7].
- The switch component reporting the flow rules in the switch to the SSA is not compromised. For instance, the components that are used for reporting on the flow rules in the switches can be protected by running them in SGX enclaves [5].

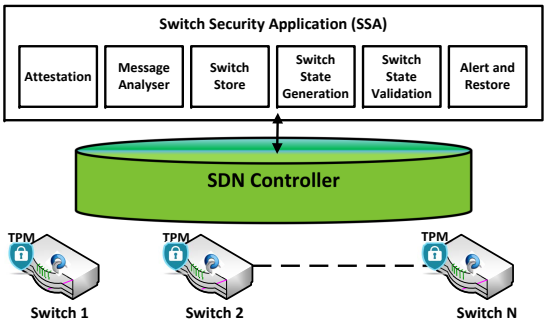

Fig. 1: Logical Architecture of Switch Security Application

The Switch Security Application (SSA) is used for detecting attacks on the switches. As shown in Figure 1, we have designed and developed the SSA as an Application running on top of an SDN Controller for flexibility reasons. The SSA maintains the details of the features or functionality of all the switches in the network.

The SDN Controller has global knowledge of all the devices in the network. Since the SSA is implemented on the SDN Controller, it is able to access the information available in the Controller and use it for monitoring and detecting the attacks on the switches. Now let us consider the operation of the SSA and discuss in detail the important components of the SSA.

A. Operation:

The TPM measures all the hardware and software components of the switch during boot time using hashes and store them securely. When the switch startup is successful, it sends OFTP_Hello message to the Controller which is subsequently forwarded to the SSA.

The SSA initiates the TPM attestation process to validate the boot time state of the switches. The SSA permits configuration of flow rules on the switches by the Controller only if the TPM attestation of the switch is successful. Furthermore, SSA validates the flow rule configuration messages for conflicts and stores all the communication between the Controller and switches. If there are any conflicts with the flowrule configuration messages, then the flowrules with higher priority are configured in the switch.

If the TPM attestation of the switch is not successful, then SSA raises an alert to the SDN administrator. This process ensures that only the switches that are found to be trusted during boot time are used for forwarding packets in SDN network. However, as discussed earlier the attacker can use different techniques to maliciously insert or delete or alter the flow rules in the switches during runtime. Let us discuss how SSA detects such malicious flow rules in the switches during runtime.

The SSA queries the switches in the data plane to report the flow rules in their flow tables. The switches generate a switch_state_report by extracting the flow rules configured in

the flow table and update the SSA. Note that the component that is generating the switch_state_report in the switch is assumed to be trusted. Hence, switch_state_report includes all unauthorised changes to the previously configured rules such as deletion or modification of the rules and any new rules inserted by the attacker.

Since the SSA has the information related to the functionality of the switch and all the authorised flow rules configuration messages from the Controller to the switches, it is able to generate a report on the expected state of the switch by querying its database. Since we assume the Controller is trusted, the report on the expected state of the switch generated by the SSA can be trusted. Now SSA compares its report on the expected state of the switch with the switch_state_report. If there are any additional rules in the switch_state_report that were not configured by the SDN Controller, or if there is mismatch with any of the rules previously configured by the Controller, then the attack on the switch is detected. Now the SSA generates an alert to the SDN administrator.

As shown in Figure 1, the logical architecture of SSA consists of different modular components such as Attestation, Message Analyser, Switch Store, Switch State Generation, Switch State Validation, and Alert and Restore for detecting the attacks on the switches. The attestation module is used for validating the boot time state of the switches. The Message Analyser is used for validating the communications between the Controller and the switches and storing them in Switch Store. The Switch Store is used for maintaining the information related to different features or functionality of the switches and storing all the communications between the Controller and switch. Switch State Generation is used for generating trusted report on the expected state of the switches by querying the Switch Store database. Switch State Validation is used for detecting the attacks by querying the switches for the switch_state_report and comparing them with the trusted report. Alert and Restore component is used for raising alerts to the SDN administrators when attacks are detected on switches and restoring the flowrules in switches. In the following subsection we will present a detail discussion on these components.

B. Attestation:

This module uses the TPM attestation between the Switches and the Controller to ensure that the switches are in trusted state during boot time. To enable in the process of attestation, all hardware and software components in the trusted platform are measured using hash values at the time of boot and measurements are stored securely to prevent modification. When a third party presents an attestation request, the trusted platform provides an attestation report in response. This response includes the measured hash values, and a set of expected hash values that are supported in the form of measurement certificates isssued by trusted certifiers. The basic idea is that a match between the measured and expected values usually indicates that the components are in a known and trusted state. In our approach, the attestation module on the SSA requests an attestation report from the physical and virtual switches and validates the report from the trusted switches.

The Controller will send flow configuration messages to the switches only if there is a match between the measured and expected values for the switch. If there is any variation in the reported values compared to the expected value then the switch is considered to be compromised and an alert is raised to the administrator. However this process only ensures that the switches are in trusted state during boot time. There is possibility for the attackers to alter the flow rules in the switches during run time. Hence there is also a need to validate the flow rules enforced in the switches during run time for detecting the attacks.

C. Message Analyser:

This component is used for analysing, validation and storing (in switch store) the communication between the Controller and the switches. OpenFlow supports different types of messages [8] between the Controller and the Switches such as Hello, Echo request, and Error. The message analyser is used for extracting the messages related to the flow rule configuration in the switches such as flow_mod, group_mod, table_mod from the Controller. Then the messages are validated to prevent conflicts in the flow rules enforced in the switches. If a new flow rule conflicts with the existing flow rules in the switch, then the flow rule with highest priority is configured in the switch. One of the challenges is that the the messages sent by the Controller can be executed in arbitrary order by the switch. Hence, we make use of Barrierreq and BarrierRes to achieve synchronization between the flow rule configuration messages issued by the Controller and the flow rule configuration messages executed on the switch.

D. Switch Store:

The switch store is used for maintaining fine granular details on the functionality or features of the switches in the network and also for storing all the communication between the Controller and switches. For instance, it is used for storing the information such as vendor information, hardware information, memory, number of interfaces, maximum number of flow rules, timeout for the flow rules, location of switch in the network topology and the flow rules configured by the Controller. The stored information is useful for generating a trusted report on the expected state of the switch. When a new switch is deployed in the network, the switch details such as number of flowtables and flowrules supported by the switch, buffer capacity for storing the messages or packets can be manually entered into the database by the administrator. We can also make use of the FeatureReq message to determine the hardware or software resources and functionality of the switch. After a secure channel (TLS) is established between the switch and Controller, the SSA will send a FeatureReq to the switch over the transport channel. The feature request consists of just an OpenFlow header message, with the FeatureReq value set in the type field. The switch will then respond to this request

with its specification and functionalities. This basic sequence does not change across OF versions.

E. Switch State Generation:

This component is used for generating trusted report on the expected state of the switches. The flow rules that are currently enforced in the switch depend on the available resources and configuration of the switches. One of the challenges in generating the expected state of the switches is that the switches support different functionality and the state of the switch can vary on different factors such as resources available at the switches. For instance, switches have very limited memory and can only accommodate few rules in their flow tables. When the flow table is full and new flow rules have to be inserted into the flow table, some of the existing rules must be removed from the switch based on different algorithms such as First-In-First-Out (FIFO) and least used. As already discussed, all the information relating to the available resources and configuration of the switch are maintained in the switch store component. Hence Switch State Generation component is able to generate the expected state of the switch by querying the switch store component. For example, consider that N is the maximum number of rules that can be stored in the switch memory and FIFO is used for storing the flow rules. In this case, the switch is expected to have rules that were configured from last N flow_mod messages. Since SSA has the information related to each switch, it can query the switch store database to determine the last N configuration messages from the Controller and generate a trusted report on the expected state of the switches.

F. Switch State Validation:

This component validates the flow rules currently enforced in the switches and detects the attacks. The component requests the switches to report the flow rules currently enforced in the switches. When the switch responds with the switch_state_report, it compares this report with the trusted state report generated by the Switch State Generation component. If there is no variation in the currently enforced flow rules compared to expected flow rules then the switch is considered to be in normal state. If there is any variation in the currently enforced flow rules compared to expected flow rules then the switch is considered to be under attack.

G. Alert and restore:

This component generates an alert to the security administrator when attacks are detected on the switches. The administrator can conduct an offline analysis to determine the seriousness of the event and decide to either isolate the victim switch from the network or restore the state of the switch with the expected state report generated by the Switch State Generation module.

IV. Implementation

We have used ONOS as the SDN Controller and Open Virtual Switches to justify our switch security application. The machine we are using for simulation is a Dell Power Edge M640 Server, consisting of two Intel Xeon Gold 6126 @ 2.6 GHz processors and 384 GB(6x64) of RAM.

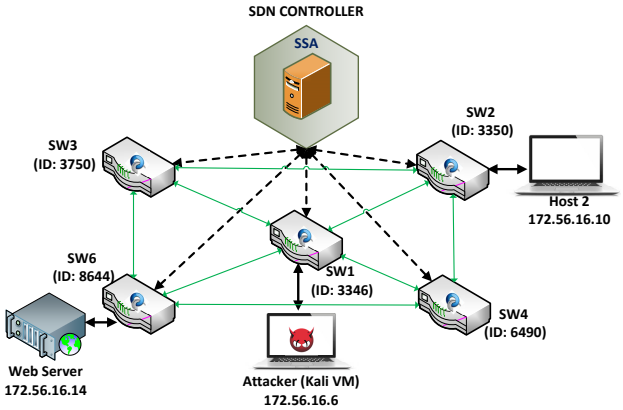

Fig. 2: Network Setup

We have used Oracle VM Box to create the network environment. We have developed the Switch Security logical architecture components as an ONOS application running over North Bound Interface. Our network configuration is shown in Figure 2.

The operation of the SSA for legitimate scenario is shown in Figure 3 where VIM diff tool is used for comparing the switch_state_report with the trusted report. In this case the Switch State Validation component in the SSA first sends a request to the switch SW1 (ID=3346) in Figure 2 to report the flow rules. The switch_state_report from switch SW1 is shown in the top Window A in Figure 3. Now, Switch State Validation component invokes the Switch State Generation component to generate a trusted report on the expected state of the switch SW1 by querying the Switch Store database for switch ID=3346. The trusted report on the expected state of the switch is shown in the below Window B in Figure 3. The Switch State Validation component compares these reports using VIM diff tool. As shown in Figure 3 the switch_state_report from SW1 matches with the trusted report. Hence the Switch State Validation component determines that the state of the switch SW1 is normal.

Now consider that an attacker has initiated attack on the switch SW1 to alter some of the flow rules as shown in Figure 2. In this case, we used kali linux machine to generate malicious flow_mod message to insert an unauthorised flow rule in SW1.

At a later time, the Switch Validation Component requests the switch to report the flow rules. The process is same as discussed earlier. However, in this case the switch_state_report also includes the flow rule inserted by the attacker as shown in top Window A in Figure 4. The Switch State Validation component detects this malicious flow rule when it compares the switch_state_report with the trusted report shown in below Window B in Figure 4. So it invokes the alert and restore component. The alert and restore component raises an alert to the SDN administrator.

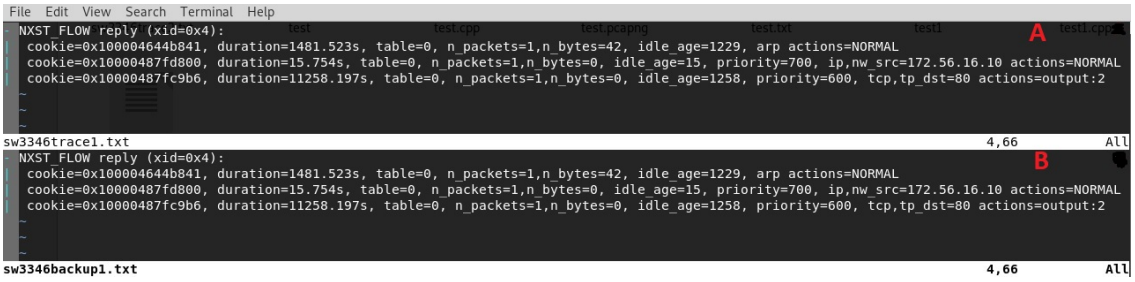

Fig. 3: A) Switch Report; B) Trusted report

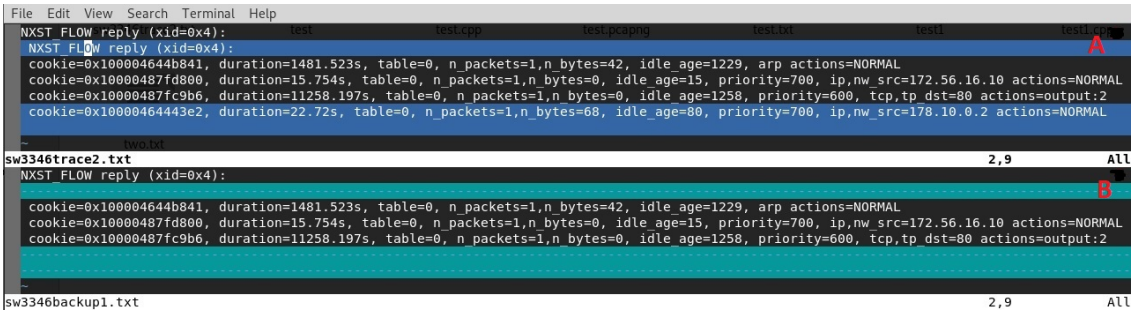

Fig. 4: A) Switch Report; B) Trusted report

V. Discussion

A. Protection for Flow Reporting Component:

In the current work, we assumed that the component that is used for generating the switch_state_report in the switch is not compromised. In the future work, we aim to use SGX enclaves [5] for protecting the code that is used for reporting the current flow rules enforced in switches. SGX introduces a trusted execution environment called enclave to shield code and data with on-chip security engines. SGX is based on extensions of the Instruction Set architecture and changes to memory access that enables application components to maintain data confidentiality and integrity, even when the hardware and all privileged software, including the OS, hypervisor and BIOS, are controlled by an untrusted entity. SGX also includes functionality for remote attestation, allowing to remotely assess the integrity of an enclave. Hence, with SGX's security guarantee, the proposed technique can be considered as robust even when the the adversary can gain full control of the switch OS or the hypervisor that is hosting the virtual switches with only exception that the code running in the enclave is secure.

to identify all the switches that are under attack. This process

B. Attack detection:

The Switch State Validation can be invoked at regular intervals or when some events occur in the network. For example, the Switch State Validation can be invoked when some of the routes are experiencing unacceptable delays or packet drops or when there is violation of SLAs. It is not an easy task for the administrator to manually validate the flow rules in all the switches and determine the switch that is under attack. Furthermore, the attacker could have performed such attacks on multiple switches and there is a need for the administrators can be very challenging in cloud environments where there can be several hundreds of virtual switches enabling communication between different tenant virtual machines. Hence, our techniques can simplify the tasks of the administrator for detecting successful attacks on the switches.

VI. Conclusion

In this paper we proposed and developed a Switch Security Application for SDN Controllers to detect attacks on switches. We presented the operation of SSA and discussed in detail the SSA components for validating the bootime state of the switches using remote attestation and validating runtime state of the switches using trusted logs for detecting the attacks. The SSA design is based on modular components and we have provided a detail discussion on the important components of SSA. Finally, we developed a prototype of SSA for ONOS SDN Controller.

Acknowledgement: The authors would like to thank the NGTF Cyber DST program for supporting this Project.

References

[1] S. Scott-Hayward *et al.*, "Sdn security: A survey," in *2013 IEEE SDN For Future Networks and Services*.
[2] C. Yoon *et al.*, "Flow wars: Systemizing the attack surface and defenses in software-defined networks," *IEEE/ACM TON*, vol. 25, no. 6, 2017.
[3] M. C. Dacier *et al.*, "Security challenges and opportunities of software-defined networking," *IEEE Security Privacy*, 2017.
[4] P. Chaignon *et al.*, "Oko: Extending open vswitch with stateful filters," in *Proc. of the Symp. on SDN Research*. ACM, 2018, p. 13.
[5] N. Paladi *et al.*, "Trust anchors in software defined networks," in *ESORICS*. Springer, 2018, pp. 485–504.
[6] K. Thimmaraju *et al.*, "Taking control of sdn-based cloud systems via the data plane," in *Symp. on SDN Research*. ACM, 2018, p. 1.
[7] T. TPM, "Main part 1 design principles specification version 1.2," 2003.
[8] *OpenFlow Switch Specification*.